📚 جزوه سیستم عامل | فصل ۸: بن‌بست (Deadlock)

📝 بخش ۶: اجتناب از بن‌بست (Deadlock Avoidance)

۱. تفاوت "پیشگیری" با "اجتناب" (خیلی مهم) 💡

در بخش قبلی (پیشگیری)، ما قوانین سفت و سختی می‌گذاشتیم (مثلاً: "همه منابع را اول بگیر" یا "به ترتیب بگیر"). این کار جلوی بن‌بست را می‌گرفت اما **کارایی سیستم را پایین می‌آورد**.

در روش **اجتناب (Avoidance)**، ما محدودیت الکی نمی‌گذاریم. در عوض، سیستم **هوشمندانه** عمل می‌کند:

- هر نخ باید از قبل اعلام کند **"حداکثر"** چقدر منبع نیاز دارد.
- هر بار که نخی منبع خواست، سیستم‌عامل بررسی می‌کند: **"آیا اگر این منبع را بدهم، در آینده به مشکل می‌خوریم؟"**
  - اگر امن بود -> می‌دهد.
  - اگر خطرناک بود -> نخ باید صبر کند (حتی اگر منبع الان آزاد باشد!).

۲. مفهوم "حالت امن" (Safe State) 🛡

سیستم در حالت امن است اگر **یک ترتیبی** (دنباله امن) وجود داشته باشد که بتوانیم کار **همه** نخ‌ها را راه بیندازیم.

- **دنباله امن (Safe Sequence):** صفی از نخ‌ها $<T_1, T_2, ..., T_n>$ است که در آن:
  - نیازهای T_i بتواند توسط (منابع آزاد فعلی + منابعی که نخ‌های قبلی $T_{j<i}$ پس می‌دهند) برآورده شود.
  - به زبان ساده: "اولی کارش تموم میشه، منابعش رو میده به دومی، دومی تموم میشه میده به سومی..."

نکته طلایی امتحانی: 🌟

- اگر سیستم در **حالت امن** باشد $\leftarrow\$ **قطعاً بن‌بست نداریم.**
- اگر سیستم در **حالت ناامن (Unsafe)** باشد $\leftarrow\$ **ممکن است** بن‌بست رخ دهد (الزاماً بن‌بست نیست، ولی خطرناک است).

۳. الگوریتم‌های اجتناب (دو روش اصلی)

بسته به نوع منابع، از دو الگوریتم متفاوت استفاده می‌کنیم:

الف) اگر هر منبع فقط ۱ نمونه داشته باشد:

✅ **الگوریتم گراف تخصیص منبع (RAG Algorithm)**

در اینجا یک یال جدید اضافه می‌کنیم: **یال ادعا (Claim Edge)** (با خط‌چین).

- $T_i \rightarrow R_j$ (خط‌چین): یعنی T_i ممکن است در آینده این منبع را بخواهد.
- **قانون:** وقتی نخی منبعی را درخواست کرد، خط‌چین را به خط پر تبدیل می‌کنیم. اگر این کار باعث ایجاد **دور (Cycle)** در گراف شد، یعنی حالت **ناامن** است و نباید منبع را بدهیم.

ب) اگر منابع دارای چند نمونه باشند (مثلاً ۳ پرینتر):

✅ **الگوریتم بانکدار (Banker's Algorithm)** 🏦 *(چون بانک‌ها هم همین‌طور وام می‌دهند: جوری وام می‌دهند که مطمئن باشند پول نقد برای راضی کردن بقیه مشتریان باقی می‌ماند).*

۴. تشریح دقیق الگوریتم بانکدار (Banker's Algorithm)

برای این الگوریتم به ۴ ساختمان داده (جدول) نیاز داریم:

1. **Available (موجودی):** برداری که می‌گوید از هر منبع چقدر الان آزاد است.
2. **Max (حداکثر نیاز):** ماتریسی که می‌گوید هر نخ در کل چقدر منبع نیاز دارد (سقف آرزوها!).
3. **Allocation (تخصیص فعلی):** ماتریسی که می‌گوید الان دست هر نخ چقدر منبع است.
4. **Need (نیاز باقیمانده):** ماتریسی که می‌گوید هر نخ چقدر دیگر می‌خواهد تا کارش تمام شود.
    - فرمول مهم: `Need = Max - Allocation`

⚙️ منطق الگوریتم (خیلی ساده):

وقتی نخی درخواست منبع می‌دهد:

1. **چک ۱:** آیا درخواستش کمتر از نیازش (Need) است؟ (زیاده‌خواهی نکرده باشد).
2. **چک ۲:** آیا درخواستش کمتر از موجودی (Available) است؟ (اصلاً در انبار داریم؟).
3. **شبیه‌سازی (Simulation):**
   - فرض می‌کنیم منبع را به او داده‌ایم. (موجودی کم می‌شود، تخصیص زیاد می‌شود، نیاز کم می‌شود).
   - حالا چک می‌کنیم: **"آیا سیستم در این حالت جدید، امن است؟"** (یعنی آیا هنوز راهی هست که کار همه تمام شود؟).
   - اگر **امن** بود $\leftarrow$ درخواست را واقعاً تأیید می‌کنیم.
   - اگر **ناامن** بود $\leftarrow$ وضعیت را برمی‌گردانیم و نخ باید صبر کند.

۵. مثال عملی (حل دستی الگوریتم) ✍

فرض کنید ۵ نخ (T_0 تا T_4) و ۳ منبع (A, B, C) داریم. موجودی فعلی (Available): 2 3 3

جدول وضعیت:

نخ	Allocation (دستش هست)	Max (کل نیازش)	Need (چقدر دیگه میخواد) *(Max - Alloc)*
T0	0 1 0	3 5 7	**3 4 7**
T1	0 0 2	2 2 3	**2 2 1**
T2	2 0 3	2 0 9	**0 0 6**
T3	1 1 2	2 2 2	**1 1 0**
T4	2 0 0	3 3 4	**1 3 4**

سوال: آیا سیستم امن است؟ **پاسخ:** باید دنباله‌ای پیدا کنیم که با موجودی 2 3 3 کارشان راه بیفتد.

1. آیا موجودی 2 3 3 جوابگوی نیاز T_0 (3 4 7) است؟ ✗ خیر.
2. جوابگوی T_1 (2 2 1) چطور؟ ✓ بله! (2>2, 3>2, 3>1)
   - فرض می‌کنیم T_1 اجرا شد و تمام کرد. منابعش (0 0 2) را برمی‌گرداند.
   - موجودی جدید: 2 3 3 + 0 0 2 = **2 3 5**

   حالا با 2 3 5 کار کی راه می‌افتد؟
   - T_3 (1 1 0) ✓. اجرا می‌شود و منابعش (1 1 2) را می‌دهد.
   - موجودی جدید: 2 3 5 + 1 1 2 = **3 4 7**

حالا با 7 4 3؟

- T_0 (7 4 3) ✓. اجرا می‌شود.
- ... و به همین ترتیب همه اجرا می‌شوند.

- چون راهی پیدا شد ($<T_0, T_1, T_3, T_4, T_2, T>$)، سیستم **امن** است.

🌟 نکاتستی کنکور

1. پیچیدگی الگوریتم بانکدار: $O(m \times n^2)$ است (n تعداد نخ، m تعداد منبع).
2. اگر منابع تک‌نمونه‌ای باشند، استفاده از "گراف" سریع‌تر از "بانکدار" است.
3. یال ادعا (Claim Edge) در گراف، نشان‌دهنده دوراندیشی سیستم برای آینده است.